

Does Random Mean Secure?

Randomness, Entropy and Perfect Secrecy Through Probability and Statistics

Project Proposal

tdvnm

September 13, 2026

Abstract

Cryptography runs on randomness, but “looks random” and “is secure” are not the same thing. This project uses the probability and statistics from an introductory course to study the gap between them. In the first half I generate binary sequences from fair coins, biased coins, people asked to be random, a linear congruential generator and a cryptographic generator, and I run frequency, runs, chi-square, autocorrelation and entropy tests on all of them. In the second half I look at three simple ciphers—Caesar, XOR and the one-time pad—through Shannon’s definition of perfect secrecy, $\mathbb{P}(M = m \mid C = c) = \mathbb{P}(M = m)$, and I measure how much a ciphertext gives away about its message using the mutual information $I(M; C)$. The main experiment keeps the cipher fixed at $C = M \oplus K$ and only makes the key’s randomness worse, step by step, to see how leakage grows as the key’s entropy falls. A last experiment shows a generator that passes every test yet is fully predictable. The conclusion I expect: good randomness is necessary for secrecy, but passing tests for randomness is not enough to guarantee it.

Contents

1	Introduction	2
2	What the project will cover	2
3	Probability and statistics I will use	5
4	Sources	5
5	A first derivation	7
6	Plan of work	8

1 Introduction

Cryptography needs random numbers. Keys, nonces, session tokens, padding: all of them have to be numbers an attacker cannot guess. But people mean two different things by “random”, and it is easy to mix them up.

1. A sequence *looks* random. It has about as many zeros as ones, the runs are the right length, nothing repeats.
2. A sequence *is* unpredictable. Even knowing everything else, you cannot say what the next bit will be.

The first is a question for statistics. The second is a question for cryptography. A sequence can pass every statistical test and still be completely predictable. And the other way around: a ciphertext can look like noise and still leak information about the message inside it.

This project is about that gap. I want to study it with the tools from the probability course, and I want to check everything by simulation, not just argue it on paper. So I use small systems I can fully analyse—coin flips, a linear congruential generator, a Caesar cipher, XOR, the one-time pad—rather than real ciphers like AES, where nothing can be worked out by hand.

1.1 What I want to find out

The main question is:

If the randomness used to make an encryption key gets worse, how much more does the ciphertext reveal about the message?

Two smaller questions come before and after it:

1. How do you test, with statistics, whether a sequence behaves randomly? And what do those tests miss?
2. If a sequence passes the tests, is it safe to use as a key?

I expect the answer to the last one is no. The point of the project is to show exactly why.

2 What the project will cover

There are four parts, plus a small interactive program. Here I only say what each part does and which piece of probability it rests on. The derivations and the results go in the full report.

2.1 Part I: Randomness, and how to test for it

I model a binary sequence $b_1b_2\cdots b_n$ as independent Bernoulli trials with $\mathbb{P}(b_i = 1) = p$. An ideal source has $p = \frac{1}{2}$ and no dependence between bits. I will generate sequences from several sources and compare them:

- fair and biased coin flips (simulated),
- sequences typed by people who were asked to “be random”,
- a linear congruential generator (LCG), which just iterates $X_{n+1} = (aX_n + c) \bmod m$,
- the operating system’s cryptographic generator,
- the Blum–Blum–Shub generator [2].

Each sequence goes through a small set of tests, simplified from the NIST suite [7]:

- **Frequency test.** If the bits are fair, the number of ones is $X \sim \text{Bin}(n, \frac{1}{2})$, so $\mathbb{E}[X] = n/2$ and $\text{Var}(X) = n/4$. The normal approximation gives a p -value.
- **Runs test.** Count the runs of identical bits and compare with what independence predicts (this is the Wald–Wolfowitz idea [10]).
- **Chi-square test on pairs.** The four patterns 00, 01, 10, 11 should each show up a quarter of the time. Pearson’s statistic [6] measures how far off they are.
- **Autocorrelation.** The correlation between b_i and b_{i+k} for small k . It should be close to zero.
- **Entropy.** $H(X) = -\sum_x \mathbb{P}(x) \log_2 \mathbb{P}(x)$. It is one bit only when the source is fair and independent.

The human sequences are the interesting case. People avoid long runs and switch between 0 and 1 too often, because to them that looks “more random”. A two-state Markov chain whose probability of switching is above $\frac{1}{2}$ models this well, and the tests above should catch it.

2.2 Part II: Encryption, and what it leaks

I use three ciphers. I chose them because it is easy to see what they do to the statistics of the message.

- The **Caesar cipher** shifts every letter by the same amount. Letter frequencies survive untouched, so frequency analysis breaks it.

- **XOR encryption**, $C = M \oplus K$, with keys from sources of different quality.
- The **one-time pad**: XOR with a key that is uniformly random, independent of the message, as long as the message, and never reused.

The standard I measure them against is Shannon’s definition of perfect secrecy [9]:

$$\mathbb{P}(M = m \mid C = c) = \mathbb{P}(M = m) \quad \text{for every } m \text{ and } c.$$

In words: seeing the ciphertext tells you nothing new about the message. This is a statement about conditional probability, which is where the project touches the course most directly. I will show that the one-time pad satisfies it and the other two do not.

Failing perfect secrecy is one thing; I also want a number for *how badly* a cipher fails. For that I use entropy, conditional entropy and mutual information, from Shannon’s 1948 paper [8] in the notation of Cover and Thomas [3]:

$$I(M; C) = H(M) - H(M \mid C).$$

Perfect secrecy is the same as $I(M; C) = 0$. Anything above zero is information that leaked. All of these can be estimated from simulated (M, C) pairs.

2.3 Part III: The main experiment—making the key worse

I take a tiny message space, $M \in \{0, 1\}^2$, and give it a lopsided distribution on purpose, say $\mathbb{P}(00) = 0.4$, $\mathbb{P}(01) = 0.3$, $\mathbb{P}(10) = 0.2$, $\mathbb{P}(11) = 0.1$. I draw many messages from it and encrypt each one as $C = M \oplus K$. The cipher never changes. The only thing I change is where the key comes from.

- A. Uniform key.** Each of the four keys with probability $\frac{1}{4}$. I expect $I(M; C)$ to go to zero as I draw more samples.
- B. Biased key.** One key value much more likely than the others, say probability 0.7. I expect $I(M; C) > 0$.
- C. Sweep.** Each key bit is 1 with probability $q \in \{0.50, 0.55, \dots, 0.95\}$. For each q I record the key entropy, the test results, the ciphertext distribution, $H(M \mid C)$ and $I(M; C)$.

Part C gives the main figure of the project: leakage plotted against key bias, and against key entropy. Because the system is only two bits, I can also compute the mutual information exactly and check the simulation against it.

2.4 Part IV: Looking random is not the same as being secure

Then I go back to the generators from Part I. An LCG with well-chosen constants passes every test in my set. But its whole future is fixed by its current state, and you can recover that state from a few outputs. So it looks random and is not secure. The Blum–Blum–Shub generator [2] is the contrast: it was built so that predicting the next bit is as hard as a number-theory problem, not just so that the output looks nice. Knuth’s chapter on random numbers [5] and Katz and Lindell’s textbook [4] both make this distinction, and this part follows them.

2.5 The interactive part

Alongside the report I will write a small program (command line or a web page). It lets you

- type in a binary sequence and see all the tests run on it,
- try to “be random” yourself and get compared to a machine,
- look at a few sequences and guess which one was really random,
- encrypt a message with each cipher and look at the ciphertext’s statistics,
- play with the leakage-versus-bias curve from Part III.

3 Probability and statistics I will use

I want to *use* the course material, not repeat it. Table 1 lists what the project needs and where Blitzstein and Hwang [1], the course textbook, cover it. Entropy and mutual information are not in that book; for those I go to Shannon [8] and Cover and Thomas [3].

Hypothesis tests and p -values run through all of Part I. The textbook does not treat them on their own, so the report will state the one definition it needs—the probability, if the source really were random, of a statistic at least as extreme as the one I saw—and rely on the distributions above.

4 Sources

The project leans on a small number of sources. Here is what I take from each one.

Concept	Where I use it	Blitzstein & Hwang
Sample spaces; naive definition of probability	Message space, key space, counting the keys that send m to c	§1.2–1.3, §1.6
Conditional probability; Bayes’ rule; law of total probability	Definition of perfect secrecy; what you believe about the message after seeing the ciphertext; ciphertext distribution as a mixture over keys	§2.2–2.3
Independence of events and of random variables	Key independent of message; independent bits in an ideal source	§2.5, §3.8
Random variables and PMFs	Everything	§3.1–3.2
Bernoulli and Binomial	One bit; number of ones in n bits (frequency test)	§3.3
Discrete Uniform	The one-time-pad key	§3.5
Expectation; linearity; indicator variables	Expected number of ones; expected number of runs (a sum of indicators)	§4.1–4.2, §4.4
Variance	Spread of the frequency and runs statistics when the source is fair	§4.6
Joint, marginal and conditional distributions	The joint law of (M, K, C) ; $H(M C)$	§7.1
Covariance and correlation	Lag- k autocorrelation of a bit sequence	§7.3
Multinomial	Counts of the pairs 00, 01, 10, 11 that feed the chi-square test	§7.4
Law of large numbers	Why estimated frequencies and entropies settle down as n grows	§10.2
Central limit theorem	The normal approximation behind the frequency and runs p -values	§10.3
Chi-square distribution	Null distribution of Pearson’s statistic	§10.4
Markov chains	A model of a person who switches between 0 and 1 too often	§11.1
Simulation	Every experiment is a Monte Carlo estimate	the “R” section of each chapter

Table 1: Probability concepts the project uses, and where the course textbook covers them.

Shannon, *Communication Theory of Secrecy Systems* (1949) [9]. The definition of perfect secrecy, the proof that the one-time pad has it, and the result that perfect secrecy needs a key with at least as much entropy as the message. The whole second half of the project rests on this paper.

Shannon, *A Mathematical Theory of Communication* (1948) [8]. Entropy, conditional entropy and mutual information. These are the numbers I use to measure leakage.

Cover and Thomas, *Elements of Information Theory* (2006) [3]. The modern notation and the basic identities I need in derivations: the chain rule, $I(M; C) = H(M) - H(M | C)$, and the fact that mutual information is never negative.

NIST SP 800-22 (2010) [7]. My frequency, runs and pair-frequency tests are simplified versions of tests in this suite. I follow its conventions for p -values.

Blum, Blum and Shub (1986) [2]. The generator I use as the example of real unpredictability, as opposed to merely looking random.

Knuth, *The Art of Computer Programming, Vol. 2, Chapter 3* (1997) [5]. Linear congruential generators, how to choose their constants, and the classical empirical tests for randomness.

Katz and Lindell, *Introduction to Modern Cryptography* (2020) [4]. The textbook treatment of perfect secrecy, and the argument for why “passes the statistical tests” is not a definition of security.

Pearson (1900) [6] and Wald and Wolfowitz (1940) [10]. The original papers for the chi-square goodness-of-fit test and the runs test.

Blitzstein and Hwang, *Introduction to Probability* (2015) [1]. The course textbook. All the probability background is referred to it, as in Table 1.

5 A first derivation

To check that the notation above actually works, here is the one derivation the whole project hangs on, done for the smallest case I will use.

Take $M, K, C \in \{0, 1\}^n$ with $C = M \oplus K$, and let K be uniform on $\{0, 1\}^n$ and independent of M . Fix any m and c . Because XOR is its own

inverse, the event $\{M = m, C = c\}$ is the same as $\{M = m, K = m \oplus c\}$, so by independence

$$\mathbb{P}(C = c \mid M = m) = \mathbb{P}(K = m \oplus c) = 2^{-n}.$$

The right-hand side does not depend on m . By the law of total probability,

$$\mathbb{P}(C = c) = \sum_{m'} \mathbb{P}(C = c \mid M = m') \mathbb{P}(M = m') = 2^{-n} \sum_{m'} \mathbb{P}(M = m') = 2^{-n},$$

and Bayes' rule finishes it:

$$\mathbb{P}(M = m \mid C = c) = \frac{\mathbb{P}(C = c \mid M = m) \mathbb{P}(M = m)}{\mathbb{P}(C = c)} = \frac{2^{-n} \mathbb{P}(M = m)}{2^{-n}} = \mathbb{P}(M = m).$$

That is Shannon's condition exactly, and it holds for *every* message distribution, including the lopsided one in Part III. The three tools used are conditional probability, the law of total probability and Bayes' rule, all from Chapter 2 of the course textbook.

The same computation shows what goes wrong when the key is not uniform. If $\mathbb{P}(K = k)$ varies with k , then $\mathbb{P}(C = c \mid M = m) = \mathbb{P}(K = m \oplus c)$ varies with m , so the ciphertext distribution depends on the message and $\mathbb{P}(M = m \mid C = c) \neq \mathbb{P}(M = m)$ in general. Part III measures *how much* it differs, using $I(M; C)$.

6 Plan of work

Table 2 is the order I intend to do things in. The simulations come before the writing in each part, so that the report describes results I actually have.

7 The end goal

I want to follow the chain

$$\text{randomness} \longrightarrow \text{uncertainty} \longrightarrow \text{information} \longrightarrow \text{secrecy}$$

from one end to the other, with each link backed by both a derivation and a simulation. Concretely, I expect to show that:

1. sequences people make up by hand are detectably not random;
2. simple statistical tests catch biased and badly built generators;
3. a Caesar cipher keeps the letter statistics of the plaintext, so its ciphertext leaks nearly all of $H(M)$;
4. the one-time pad with a uniform key gives $I(M; C) \approx 0$ in simulation, matching the exact answer of zero;

Week	Work
1	Set up the code: bit-sequence sources (coins, LCG, OS generator, BBS) and the five tests. Check each test on a known-fair source by looking at the distribution of its p -values.
2	Collect human sequences; fit the two-state Markov model; write Part I.
3	Caesar, XOR and one-time-pad implementations; estimators for $H(\cdot)$, $H(\cdot \cdot)$ and $I(\cdot; \cdot)$ from samples; write Part II.
4	Main experiment (A, B and the sweep C); exact mutual information for the two-bit system to compare against; the main figure.
5	Part IV: recover an LCG's state from its outputs; contrast with BBS. Interactive program.
6	Write the introduction and conclusion last; proof-read; final figures.

Table 2: Planned order of work.

5. biasing the key makes $I(M; C)$ positive, and the leakage keeps growing as the key's entropy falls;
6. an LCG can pass every test in my set and still be predictable from a handful of outputs.

Put together, these support the claim the project is named after:

Good randomness is necessary for secrecy. But looking random is not enough to make something secure.

References

- [1] Joseph K. Blitzstein and Jessica Hwang. *Introduction to Probability*. Texts in Statistical Science. Boca Raton, FL: CRC Press, 2015. ISBN: 978-1-4665-7557-8.
- [2] Lenore Blum, Manuel Blum, and Michael Shub. "A Simple Unpredictable Pseudo-Random Number Generator". In: *SIAM Journal on Computing* 15.2 (1986), pp. 364–383. DOI: 10.1137/0215025.
- [3] Thomas M. Cover and Joy A. Thomas. *Elements of Information Theory*. 2nd ed. Hoboken, NJ: Wiley-Interscience, 2006. ISBN: 978-0-471-24195-9.
- [4] Jonathan Katz and Yehuda Lindell. *Introduction to Modern Cryptography*. 3rd ed. Boca Raton, FL: CRC Press, 2020. ISBN: 978-0-8153-5436-9.

- [5] Donald E. Knuth. *The Art of Computer Programming, Volume 2: Seminumerical Algorithms*. 3rd ed. Reading, MA: Addison-Wesley, 1997. ISBN: 978-0-201-89684-8.
- [6] Karl Pearson. “On the Criterion that a Given System of Deviations from the Probable in the Case of a Correlated System of Variables is such that it can be Reasonably Supposed to have Arisen from Random Sampling”. In: *Philosophical Magazine, Series 5* 50.302 (1900), pp. 157–175. DOI: 10.1080/14786440009463897.
- [7] Andrew Rukhin et al. *A Statistical Test Suite for Random and Pseudorandom Number Generators for Cryptographic Applications*. Tech. rep. Special Publication 800-22, Revision 1a. National Institute of Standards and Technology, 2010. DOI: 10.6028/NIST.SP.800-22r1a.
- [8] Claude E. Shannon. “A Mathematical Theory of Communication”. In: *Bell System Technical Journal* 27.3–4 (1948), pp. 379–423, 623–656. DOI: 10.1002/j.1538-7305.1948.tb01338.x.
- [9] Claude E. Shannon. “Communication Theory of Secrecy Systems”. In: *Bell System Technical Journal* 28.4 (1949), pp. 656–715. DOI: 10.1002/j.1538-7305.1949.tb00928.x.
- [10] Abraham Wald and Jacob Wolfowitz. “On a Test Whether Two Samples are from the Same Population”. In: *Annals of Mathematical Statistics* 11.2 (1940), pp. 147–162. DOI: 10.1214/aoms/1177731909.